JavaScript is disabled on your browser.



compact1, compact2, compact3

java.time.temporal

## Class TemporalAdjusters

java.lang.Object
    java.time.temporal.TemporalAdjusters

---

```
public final class TemporalAdjusters
extends Object
```

Common and useful TemporalAdjusters.

Adjusters are a key tool for modifying temporal objects. They exist to externalize the process of adjustment, permitting different approaches, as per the strategy design pattern. Examples might be an adjuster that sets the date avoiding weekends, or one that sets the date to the last day of the month.

There are two equivalent ways of using a `TemporalAdjuster`. The first is to invoke the method on the interface directly. The second is to use `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisAdjuster.adjustInto(temporal);
temporal = temporal.with(thisAdjuster);
```

It is recommended to use the second approach, `with(TemporalAdjuster)`, as it is a lot clearer to read in code.

This class contains a standard set of adjusters, available as static methods. These include:

- finding the first or last day of the month
- finding the first day of next month
- finding the first or last day of the year
- finding the first day of next year

- finding the first or last day-of-week within a month, such as "first Wednesday in June"
- finding the next or previous day-of-week, such as "next Thursday"

**Implementation Requirements:**

All the implementations supplied by the static methods are immutable.

**Since:**

1.8

**See Also:**

TemporalAdjuster

## *Method Summary*

**Concrete Methods**

| Modifier and Type | Method |
| --- | --- |
| static TemporalAdjuster | dayOfWeekInMonth(int ordinal, DayOfWeek dayOfWeek) |
| static TemporalAdjuster | firstDayOfMonth() |
| static TemporalAdjuster | firstDayOfNextMonth() |
| static TemporalAdjuster | firstDayOfNextYear() |
| static TemporalAdjuster | firstDayOfYear() |

| | |
|---|---|
| static TemporalAdjuster | **firstInMonth**(DayOfWeek dayOfWeek) |
| static TemporalAdjuster | **lastDayOfMonth**() |
| static TemporalAdjuster | **lastDayOfYear**() |
| static TemporalAdjuster | **lastInMonth**(DayOfWeek dayOfWeek) |
| static TemporalAdjuster | **next**(DayOfWeek dayOfWeek) |

| | |
|---|---|
| static **TemporalAdjuster** | **nextOrSame**(**DayOfWeek** dayOfWeek) |
| static **TemporalAdjuster** | **ofDateAdjuster**(**UnaryOperator**<**LocalDate**> dateBasedAdjuste |
| static **TemporalAdjuster** | **previous**(**DayOfWeek** dayOfWeek) |
| static **TemporalAdjuster** | **previousOrSame**(**DayOfWeek** dayOfWeek) |

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## *Method Detail*

### ofDateAdjuster

```
public static TemporalAdjuster ofDateAdjuster(UnaryOperator<LocalDate> dateBasedA
```

Obtains a `TemporalAdjuster` that wraps a date adjuster.

The `TemporalAdjuster` is based on the low level `Temporal` interface. This method allows an adjustment from `LocalDate` to `LocalDate` to be wrapped to match the temporal-based interface. This is provided for convenience to make user-written adjusters simpler.

In general, user-written adjusters should be static constants:

```
  static TemporalAdjuster TWO_DAYS_LATER =
       TemporalAdjusters.ofDateAdjuster(date -> date.plusDays(2));
```

**Parameters:**

`dateBasedAdjuster - the date-based adjuster, not null`

**Returns:**

`the temporal adjuster wrapping on the date adjuster, not null`

### firstDayOfMonth

```
public static TemporalAdjuster firstDayOfMonth()
```

Returns the "first day of month" adjuster, which returns a new date set to the first day of the current month.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2011-01-01.
The input 2011-02-15 will return 2011-02-01.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  temporal.with(DAY_OF_MONTH, 1);
```

**Returns:**

the first day-of-month adjuster, not null

### lastDayOfMonth

```
public static TemporalAdjuster lastDayOfMonth()
```

Returns the "last day of month" adjuster, which returns a new date set to the last day of the current month.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2011-01-31.
The input 2011-02-15 will return 2011-02-28.
The input 2012-02-15 will return 2012-02-29 (leap year).
The input 2011-04-15 will return 2011-04-30.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  long lastDay = temporal.range(DAY_OF_MONTH).getMaximum();
  temporal.with(DAY_OF_MONTH, lastDay);
```

**Returns:**

the last day-of-month adjuster, not null

### firstDayOfNextMonth

```
public static TemporalAdjuster firstDayOfNextMonth()
```

Returns the "first day of next month" adjuster, which returns a new date set to the first day of the next month.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2011-02-01.
The input 2011-02-15 will return 2011-03-01.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  temporal.with(DAY_OF_MONTH, 1).plus(1, MONTHS);
```

**Returns:**

the first day of next month adjuster, not null

### firstDayOfYear

```
public static TemporalAdjuster firstDayOfYear()
```

Returns the "first day of year" adjuster, which returns a new date set to the first day of the current year.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2011-01-01.
The input 2011-02-15 will return 2011-01-01.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  temporal.with(DAY_OF_YEAR, 1);
```

**Returns:**

the first day-of-year adjuster, not null

### lastDayOfYear

```
public static TemporalAdjuster lastDayOfYear()
```

Returns the "last day of year" adjuster, which returns a new date set to the last day of the current year.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2011-12-31.
The input 2011-02-15 will return 2011-12-31.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  long lastDay = temporal.range(DAY_OF_YEAR).getMaximum();
  temporal.with(DAY_OF_YEAR, lastDay);
```

**Returns:**

the last day-of-year adjuster, not null

### firstDayOfNextYear

```
public static TemporalAdjuster firstDayOfNextYear()
```

Returns the "first day of next year" adjuster, which returns a new date set to the first day of the next year.

The ISO calendar system behaves as follows:
The input 2011-01-15 will return 2012-01-01.

The behavior is suitable for use with most calendar systems. It is equivalent to:

```
  temporal.with(DAY_OF_YEAR, 1).plus(1, YEARS);
```

**Returns:**

the first day of next month adjuster, not null

### firstInMonth

```
public static TemporalAdjuster firstInMonth(DayOfWeek dayOfWeek)
```

Returns the first in month adjuster, which returns a new date in the same month with the first matching day-of-week. This is used for expressions like 'first Tuesday in March'.

The ISO calendar system behaves as follows:
The input 2011-12-15 for (MONDAY) will return 2011-12-05.
The input 2011-12-15 for (FRIDAY) will return 2011-12-02.

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` and `DAY_OF_MONTH` fields and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

dayOfWeek - the day-of-week, not null

**Returns:**

the first in month adjuster, not null

### lastInMonth

```
public static TemporalAdjuster lastInMonth(DayOfWeek dayOfWeek)
```

Returns the last in month adjuster, which returns a new date in the same month with the last matching day-of-week. This is used for expressions like 'last Tuesday in March'.

The ISO calendar system behaves as follows:
The input 2011-12-15 for (MONDAY) will return 2011-12-26.
The input 2011-12-15 for (FRIDAY) will return 2011-12-30.

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` and `DAY_OF_MONTH` fields and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

dayOfWeek - the day-of-week, not null

**Returns:**

the first in month adjuster, not null

### dayOfWeekInMonth

```
public static TemporalAdjuster dayOfWeekInMonth(int ordinal,
                                                DayOfWeek dayOfWeek)
```

Returns the day-of-week in month adjuster, which returns a new date in the same month with the ordinal day-of-week. This is used for expressions like the 'second Tuesday in March'.

The ISO calendar system behaves as follows:
The input 2011-12-15 for (1,TUESDAY) will return 2011-12-06.
The input 2011-12-15 for (2,TUESDAY) will return 2011-12-13.
The input 2011-12-15 for (3,TUESDAY) will return 2011-12-20.
The input 2011-12-15 for (4,TUESDAY) will return 2011-12-27.
The input 2011-12-15 for (5,TUESDAY) will return 2012-01-03.
The input 2011-12-15 for (-1,TUESDAY) will return 2011-12-27 (last in month).
The input 2011-12-15 for (-4,TUESDAY) will return 2011-12-06 (3 weeks before last in month).
The input 2011-12-15 for (-5,TUESDAY) will return 2011-11-29 (4 weeks before last in month).
The input 2011-12-15 for (0,TUESDAY) will return 2011-11-29 (last in previous month).

For a positive or zero ordinal, the algorithm is equivalent to finding the first day-of-week that matches within the month and then adding a number of weeks to it. For a negative ordinal, the algorithm is equivalent to finding the last day-of-week that matches within the month and then subtracting a number of weeks to it. The ordinal number of weeks is not validated and is interpreted leniently according to this algorithm. This definition means that an ordinal of zero finds the last matching day-of-week in the previous month.

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` and `DAY_OF_MONTH` fields and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

`ordinal - the week within the month, unbounded but typically from -5 to 5`

`dayOfWeek - the day-of-week, not null`

**Returns:**

`the day-of-week in month adjuster, not null`

### next

```
public static TemporalAdjuster next(DayOfWeek dayOfWeek)
```

Returns the next day-of-week adjuster, which adjusts the date to the first occurrence of the specified day-of-week after the date being adjusted.

The ISO calendar system behaves as follows:
The input 2011-01-15 (a Saturday) for parameter (MONDAY) will return 2011-01-17 (two days later).
The input 2011-01-15 (a Saturday) for parameter (WEDNESDAY) will return 2011-01-19 (four days later).
The input 2011-01-15 (a Saturday) for parameter (SATURDAY) will return 2011-01-22 (seven days later).

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` field and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

`dayOfWeek - the day-of-week to move the date to, not null`

**Returns:**

`the next day-of-week adjuster, not null`

### nextOrSame

```
public static TemporalAdjuster nextOrSame(DayOfWeek dayOfWeek)
```

Returns the next-or-same day-of-week adjuster, which adjusts the date to the first occurrence of the specified day-of-week after the date being adjusted unless it is already on that day in which case the same object is returned.

The ISO calendar system behaves as follows:
The input 2011-01-15 (a Saturday) for parameter (MONDAY) will return 2011-01-17 (two days later).
The input 2011-01-15 (a Saturday) for parameter (WEDNESDAY) will return 2011-01-19 (four days later).
The input 2011-01-15 (a Saturday) for parameter (SATURDAY) will return 2011-01-15 (same as input).

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` field and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

`dayOfWeek - the day-of-week to check for or move the date to, not null`

**Returns:**

`the next-or-same day-of-week adjuster, not null`

### previous

```
public static TemporalAdjuster previous(DayOfWeek dayOfWeek)
```

Returns the previous day-of-week adjuster, which adjusts the date to the first occurrence of the specified day-of-week before the date being adjusted.

The ISO calendar system behaves as follows:
The input 2011-01-15 (a Saturday) for parameter (MONDAY) will return 2011-01-10 (five days earlier).
The input 2011-01-15 (a Saturday) for parameter (WEDNESDAY) will return 2011-01-12 (three days earlier).
The input 2011-01-15 (a Saturday) for parameter (SATURDAY) will return 2011-01-08 (seven days earlier).

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` field and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

`dayOfWeek - the day-of-week to move the date to, not null`

**Returns:**

`the previous day-of-week adjuster, not null`

### previousOrSame

```
public static TemporalAdjuster previousOrSame(DayOfWeek dayOfWeek)
```

Returns the previous-or-same day-of-week adjuster, which adjusts the date to the first occurrence of the specified day-of-week before the date being adjusted unless it is already on that day in which case the same object is returned.

The ISO calendar system behaves as follows:
The input 2011-01-15 (a Saturday) for parameter (MONDAY) will return 2011-01-10 (five days earlier).
The input 2011-01-15 (a Saturday) for parameter (WEDNESDAY) will return 2011-01-12 (three days earlier).
The input 2011-01-15 (a Saturday) for parameter (SATURDAY) will return 2011-01-15 (same as input).

The behavior is suitable for use with most calendar systems. It uses the `DAY_OF_WEEK` field and the `DAYS` unit, and assumes a seven day week.

**Parameters:**

`dayOfWeek - the day-of-week to check for or move the date to, not null`

**Returns:**

```
the previous-or-same day-of-week adjuster, not null
```



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.